UNIVERSITY OF Southampton
School of Electronics and Computer Science

Comp1202: Inheritance I

Super and Sub-classes

Coming up

- Inheritance and Code Duplication
  - Super and sub-classes
  - Inheritance hierarchies
- Inheritance and Encapsulation
  - Constructors and super()
- Inheritance, References and Collections
  - Substitution
- The Object Class
  - Inheritance at the core of Java

Inheritance and Code Duplication

Duplication in Classes

- We have seen how duplicating code is bad
  - It creates opportunities for errors (inconsistency)
  - Is harder to maintain
  - Makes code harder to read
- But consider properly encapsulated but similar classes
  - (classes that are responsible for themselves, and hide methods via the protected keyword)
  - They can’t share code with another class
  - So do you have to duplicate it?

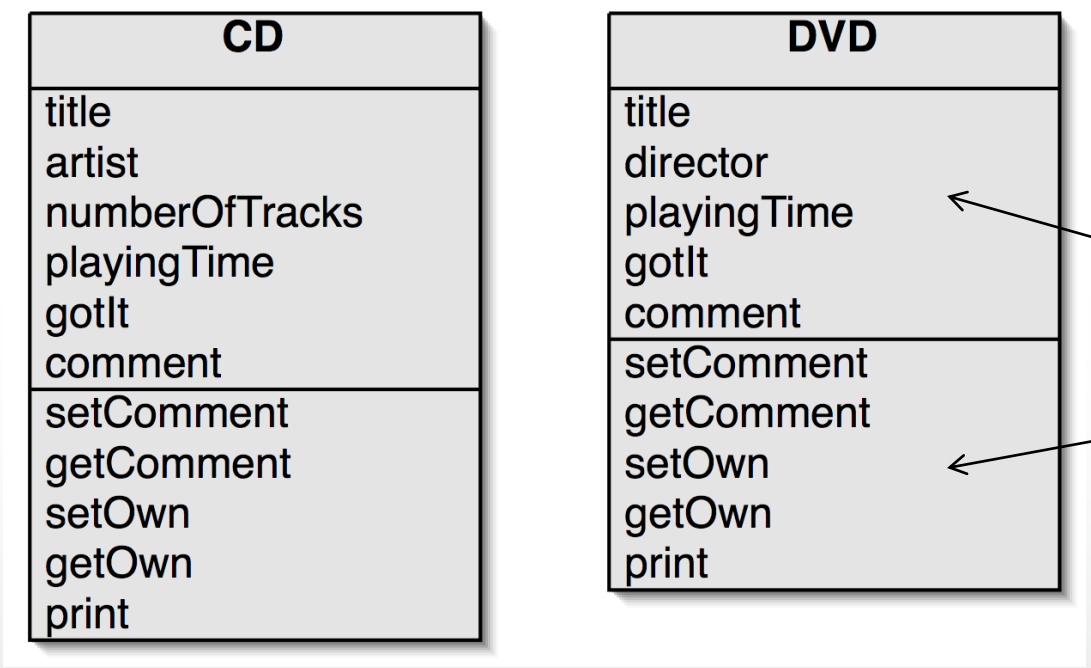
Example

- Think about music CDs and films on DVDs...

- What properties do they share?

N.B. The following example is based on the the DOME project from the BlueJ book

DoME classes



N.B This is a simple form of **class diagram**

The top half shows properties

The bottom half shows methods

DVD Source Code

```
public class DVD
{
  private String title;
  private String director;
  private String comment;

  DVD(String theTitle, String theDirector)
  {
            title = theTitle;
            director = theDirector;
            comment = " ";
  }

  void setComment(String newComment)
  { ... }

  String getComment()
  { ... }

  void print()
  { ... }
  ...
}
```

DVD Source Code

```
public class DVD
{
  private String title;
  private String director;
  private String comment;

  DVD(String theTitle, String theDirector)
  {
            title = theTitle;
            director = theDirector;
            comment = " ";
  }

  void setComment(String newComment)
  { ... }

  String getComment()
  { ... }

  void print()
  { ... }
  ...
}
```

CD Source Code

```
public class CD
{
  private String title;
  private String artist;
  private String comment;

  CD(String theTitle, String theArtist)
  {
            title = theTitle;
            artist = theArtist;
            comment = " ";
  }

  void setComment(String newComment)
  { ... }

  String getComment()
  { ... }

  void print()
  { ... }
  ...
}
```

DVD Source Code

```
public class DVD
{
   private String title;
   private String director;
   private String comment;

   DVD(String theTitle, String theDirector)
   {
            title = theTitle;
            director = theDirector;
            comment = " ";
   }

   void setComment(String newComment)
   { ... }

   String getComment()
   { ... }

   void print()
   { ... }
   ...
}
```

A kind of Code Duplication

CD Source Code

```
public class CD
{
   private String title;
   private String artist;
   private String comment;

   CD(String theTitle, String theArtist)
   {
            title = theTitle;
            artist = theArtist;
            comment = " ";
   }

   void setComment(String newComment)
   { ... }

   String getComment()
   { ... }

   void print()
   { ... }
   ...
}
```

Code Duplication

- Even this code duplication is a:

Bad Thing

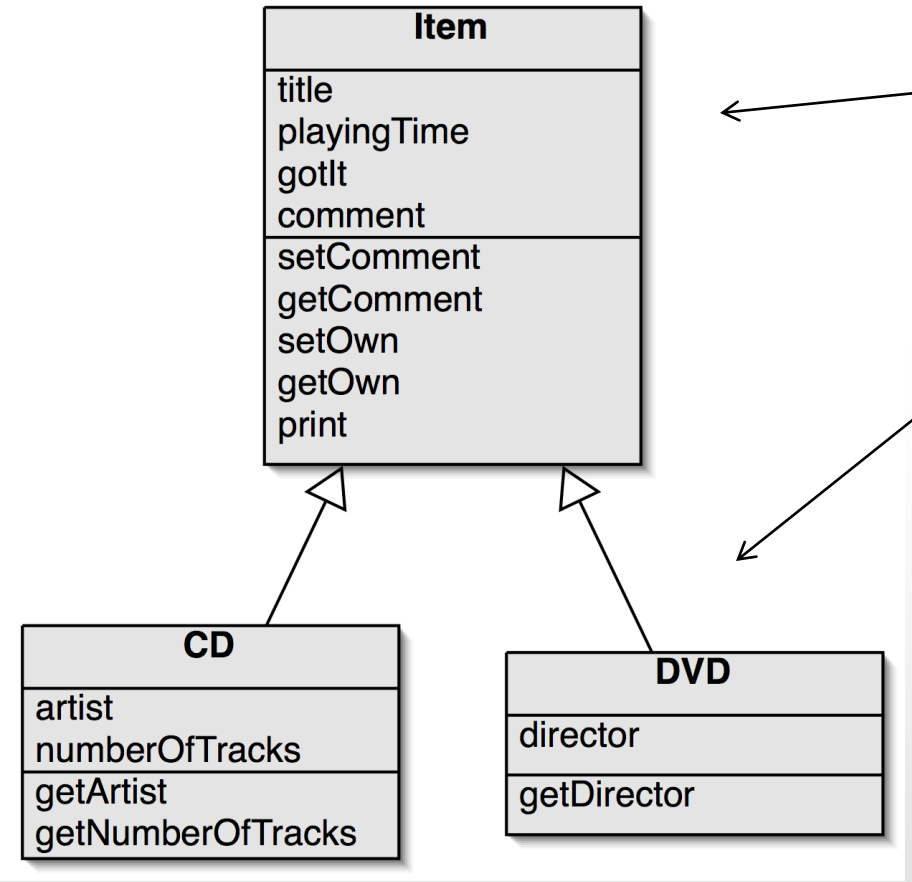
- If you debug one bit of code in one class, you have to change each duplicated code fragment individually. This can take ages!
- If you have 150 classes with the same bit of code, it’s very easy to miss one...

Wouldn’t it be good…

- If a class could inherit some properties that it shares with other classes
  - a bit like a common template
  - shipping out common functionality and keeping the specific stuff in the class

- In Object Orientated programming, this is known as **inheritance**

Using inheritance



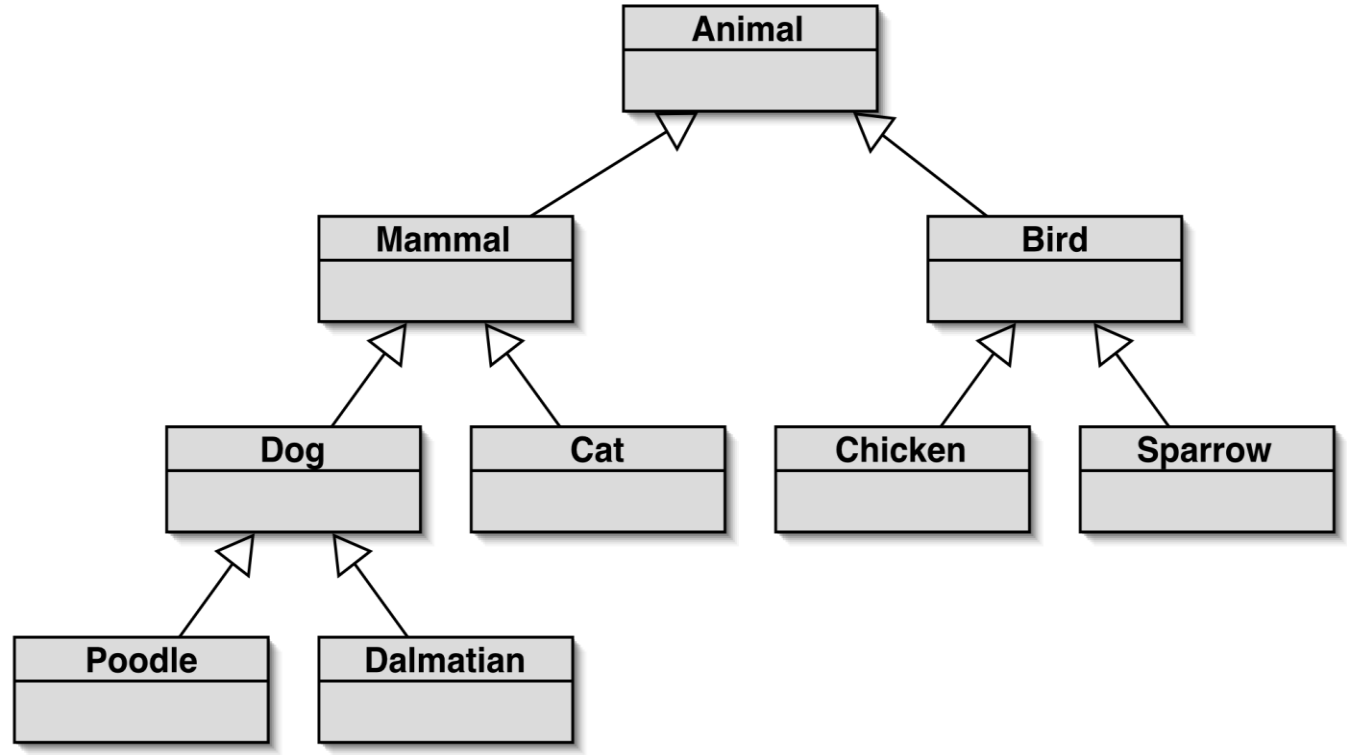
Any class that is inherited from is called a **superclass**

Any class that inherits from another is called a **subclass**

The sub-classes inherit all the properties and methods from the superclass

They can also add more of their own

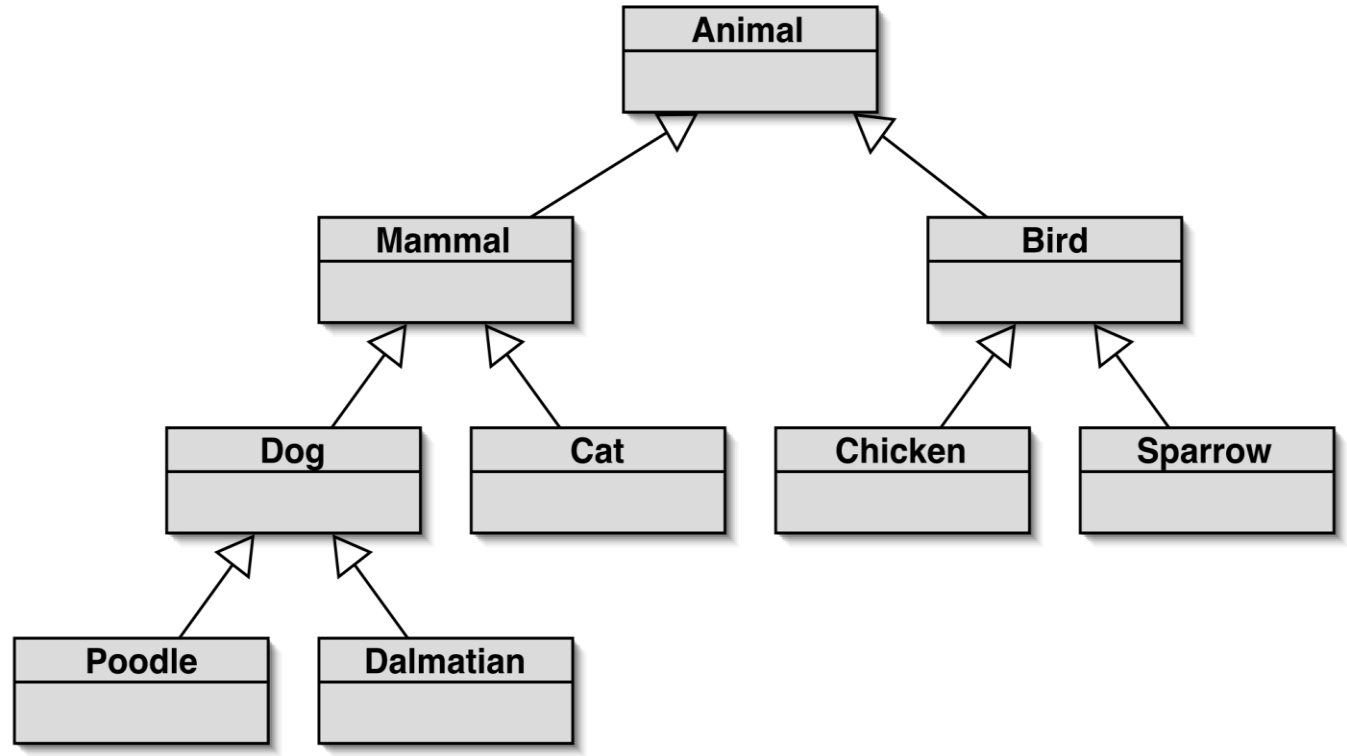
Inheritance hierarchies



Any subclass can also be a superclass

This can create a tree of classes called an **inheritance hierarchy**

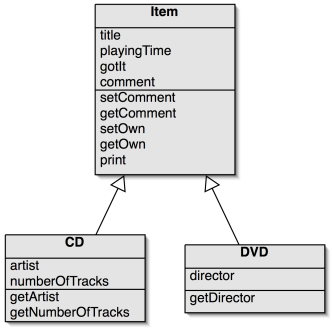
Inheritance hierarchies



Any subclass can also be a superclass

This can create a tree of classes called an **inheritance hierarchy**

N.B How class diagrams show inheritance (with the arrows pointing from sub-class to superclass)

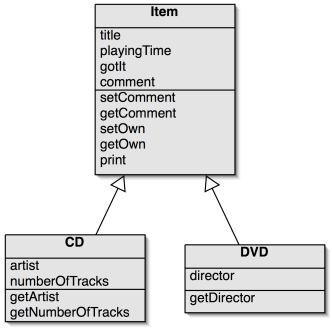


Inheritance in Java

```
public class Item
{
    private String title;
    private int playingTime;
    private boolean gotIt;
    private String comment;

    // constructors and methods omitted.
}
```

You don’t need to add anything to the superclass



Inheritance in Java

```
public class CD extends Item
{
    private String artist;
    private int numberOfTracks;

    // constructors and methods omitted.
}
```

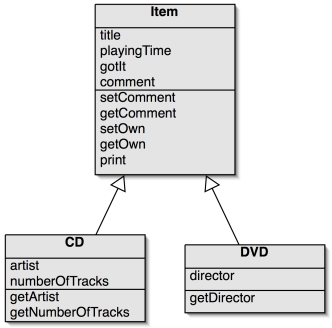
```
public class DVD extends Item
{
    private String director;

    // constructors and methods omitted.
}
```

You don’t need to add anything to the superclass

You declare the inheritance in the sub-class using the **extends** keyword

Q: What is the advantage of leaving the superclass unchanged?



Inheritance in Java

```
public class CD extends Item
{
    private String artist;
    private int numberOfTracks;

    // constructors and methods omitted.
}
```

```
public class DVD extends Item
{
    private String director;

    // constructors and methods omitted.
}
```

You don't need to add anything to the superclass

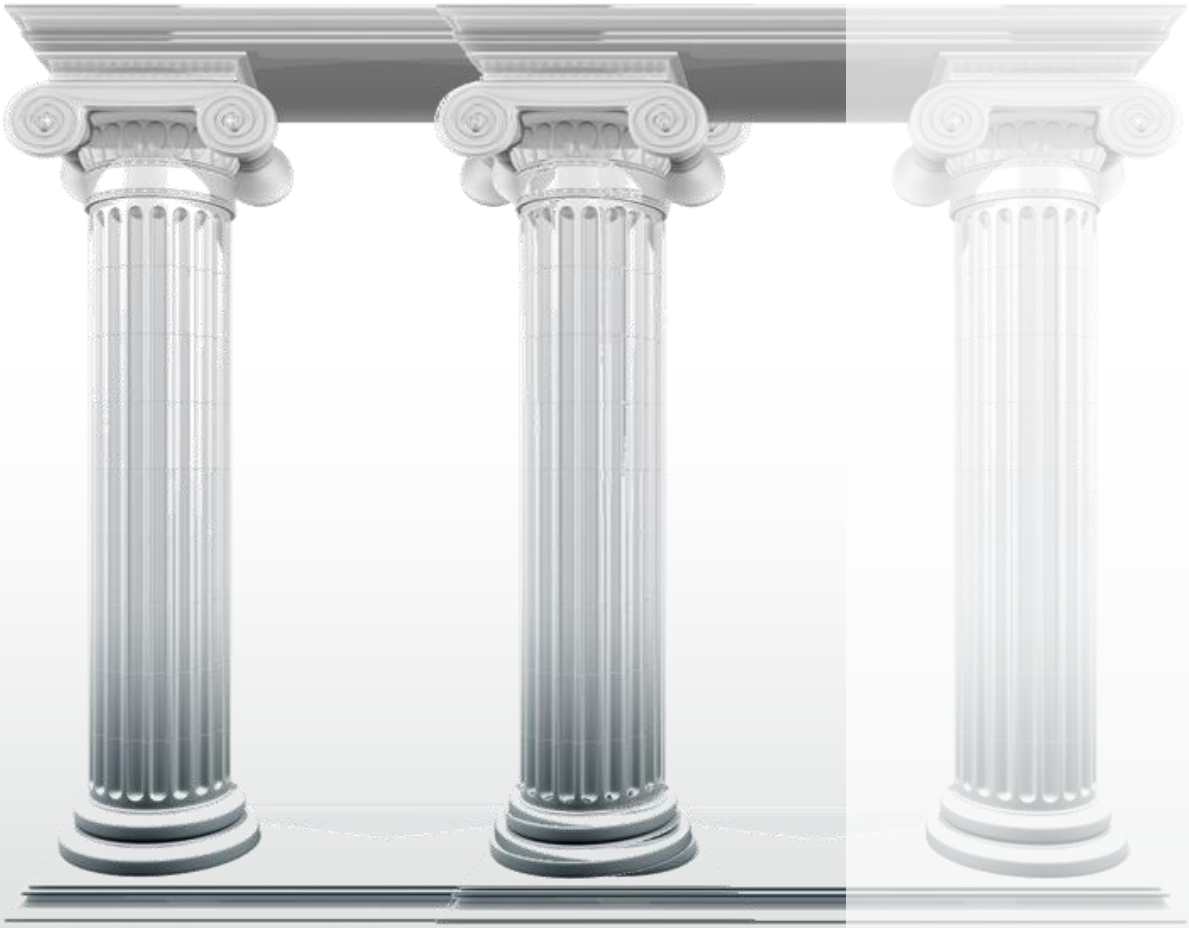
You declare the inheritance in the sub-class using the **extends** keyword

Q: What is the advantage of leaving the superclass unchanged?

A: So you can inherit from any class – even those you didn't create and/or can't edit

Inheritance and Encapsulation

The Three Pillars of OOP



Encapsulation Inheritance Polymorphism

Encapsulation and Inheritance

- Encapsulation is the principle that every class should be responsible for itself
  - We enforce it using the public and private keywords
- But is the encapsulation within the class or within the whole inheritance hierarchy?
- In other words: Should sub-classes be able to see all the properties and methods in their super classes?

Encapsulation Expanded

- In fact Java uses several keywords for encapsulation
- Public
  - Everyone can see it
- Protected
  - Only *this class, its sub-classes and the package* can see it
- Default (no keyword)
  - Only *this class and the package* can see it
- Private
  - Only *this class* can see it

Encapsulation Expanded

- In fact Java uses several keywords for encapsulation

- Public
  - Everyone can see it

- Protected
  - Only *this class, its sub-classes and the package* can see it

- Default (no keyword)
  - Only *this class and the package* can see it

- Private
  - Only *this class* can see it

Remember – Be Paranoid: assume everything should be private from the beginning

Encapsulation Expanded

- In fact Java uses several keywords for encapsulation

- Public
  - Everyone can see it

- Protected
  - Only *this class, its sub-classes and the package* can see it

- Default (no keyword)
  - Only *this class and the package* can see it

- Private
  - Only *this class* can see it

Remember – Be Paranoid: assume everything should be private from the beginning

Explicitly make public the methods (and sometime properties) you want other classes to use

Encapsulation Expanded

- In fact Java uses several keywords for encapsulation

- Public
  - Everyone can see it

- Protected
  - Only *this class, its sub-classes and the package* can see it

- Default (no keyword)
  - Only *this class and the package* can see it

- Private
  - Only *this class* can see it

Remember – Be Paranoid: assume everything should be private from the beginning

Explicitly make public the methods (and sometime properties) you want other classes to use

Explicitly make protected any implementation details you think sub-classes may need access to

Encapsulation and Constructors

- Remember Constructors
  - Special methods that initialise an object when you create it using the **new** keyword

- Constructors are important for encapsulation as a class should be in charge of initialising itself

So Are These Classes Encapsulated?

```
public class Item
{
    protected String title;
    protected int playingTime;
    protected boolean gotIt;
    protected String comment;

    public Item(String theTitle, int time)
    {
        title = theTitle;
        playingTime = time;
        gotIt = false;
        comment = "";
    }

    // methods omitted
}
```

```
public class CD extends Item
{
    protected String artist;
    protected int numberOfTracks;

    public CD( String theTitle,
               String theArtist,
               int tracks,
               int time)
    {
        title = theTitle;
        playingTime = time;
        gotIt = false;
        comment = "";
        artist = theArtist;
        numberOfTracks = tracks;
    }

    // methods omitted
}
```

So Are These Classes Encapsulated?

```
public class Item
{
    protected String title;
    protected int playingTime;
    protected boolean gotIt;
    protected String comment;

    public Item(String theTitle, int time)
    {
        title = theTitle;
        playingTime = time;
        gotIt = false;
        comment = "";
    }

    // methods omitted
}
```

```
public class CD extends Item
{
    protected String artist;
    protected int numberOfTracks;

    public CD( String theTitle,
               String theArtist,
               int tracks,
               int time)
    {
        title = theTitle;
        playingTime = time;
        gotIt = false;
        comment = "";
        artist = theArtist;
        numberOfTracks = tracks;
    }

    // methods omitted
}
```

No. Because here CD is taking the responsibility for constructing properties defined in the Item class

Better to call the super-class constructor

```
public class Item
{
    private String title;
    private int playingTime;
    private boolean gotIt;
    private String comment;

    public Item(String theTitle, int time)
    {
        title = theTitle;
        playingTime = time;
        gotIt = false;
        comment = "";
    }

    // methods omitted
}
```

```
public class CD extends Item
{
    private String artist;
    private int numberOfTracks;

    public CD( String theTitle,
               String theArtist,
               int tracks,
               int time)
    {
        super(theTitle, time);
        artist = theArtist;
        numberOfTracks = tracks;
    }

    // methods omitted
}
```

We can call the super-class' constructor using the **super** keyword. Now we can make the properties **private** and we have proper encapsulation.

Superclass constructor rules

- In fact subclass constructors must always contain a 'super' call.
- If none is written, the compiler acts as if there is one (without parameters)
  - works only if the superclass has a constructor without parameters
- Must be the first statement in the subclass constructor.

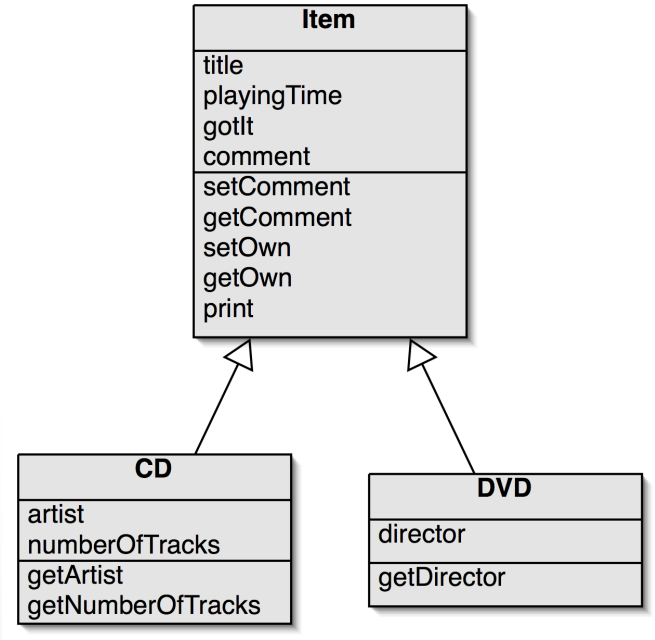
Inheritance, References and Collections

References and Inheritance

```
Item item1 = new CD("Nevermind",
                    "Nivana",
                    11, 56);

Item item2 = new DVD("Gladiator",
                     "Ridley Scott",
                     155);

item1.print();
item2.print();
```



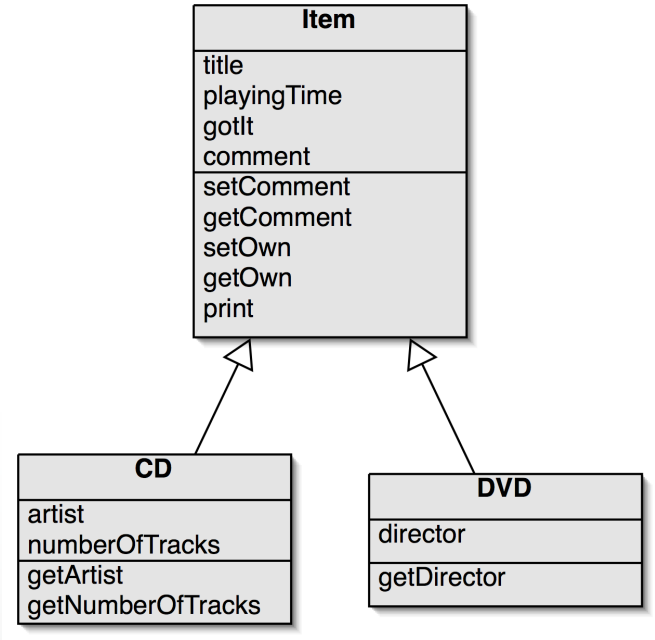
Will this code work?

References and Inheritance

```
Item item1 = new CD("Nevermind",
                    "Nivana",
                    11, 56);

Item item2 = new DVD("Gladiator",
                     "Ridley Scott",
                     155);

item1.print();
item2.print();
```



Yes, this is called **substitution**. An Item reference can store any sub-class of Item

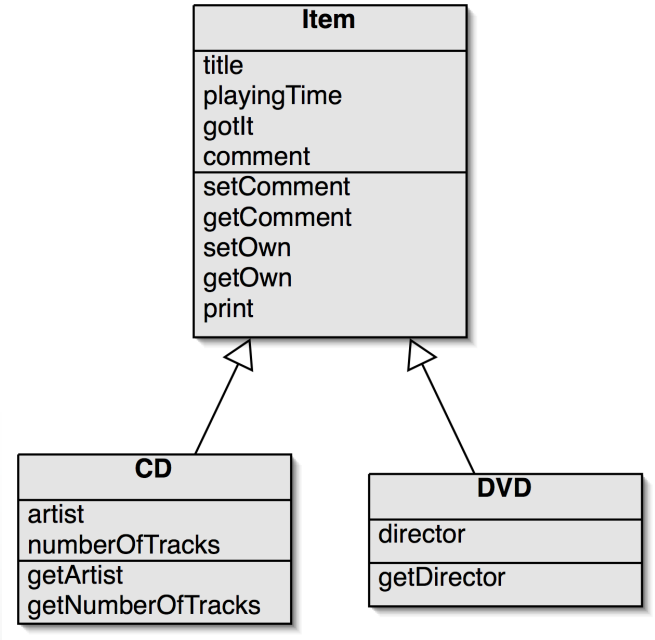
Collections and Inheritance

```
ArrayList<Item> items;
items = new ArrayList<Item>();

items.add( new CD("Nevermind",
                  "Nivana",
                  11, 56) );

items.add( new DVD("Gladiator",
                   "Ridley Scott",
                   155) );


for(Item i : items) {
         i.print();
}
```



Yes, this is called **substitution**. An Item reference can store any sub-class of Item

This is useful in collections. Because we can declare a collection of super-classes and can use it to store any type of sub-class

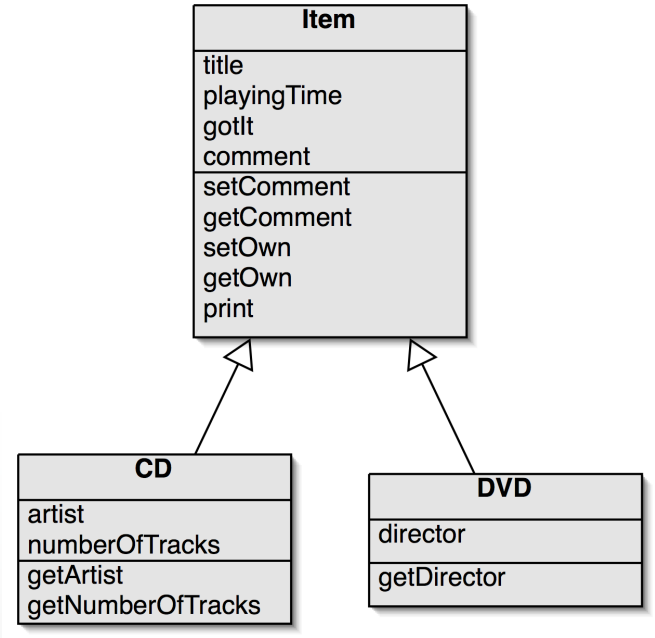
Collections and Inheritance

```
ArrayList<Item> items;
items = new ArrayList<Item>();

items.add( new CD("Nevermind",
                  "Nivana",
                  11, 56) );

items.add( new DVD("Gladiator",
                   "Ridley Scott",
                   155) );


for(Item i : items) {
        i.print();
        int tracks = i.getNumberOfTracks();
        System.out.println(tracks);
}
```



Yes, this is called **substitution**. An Item reference can store any sub-class of Item

But will this work?

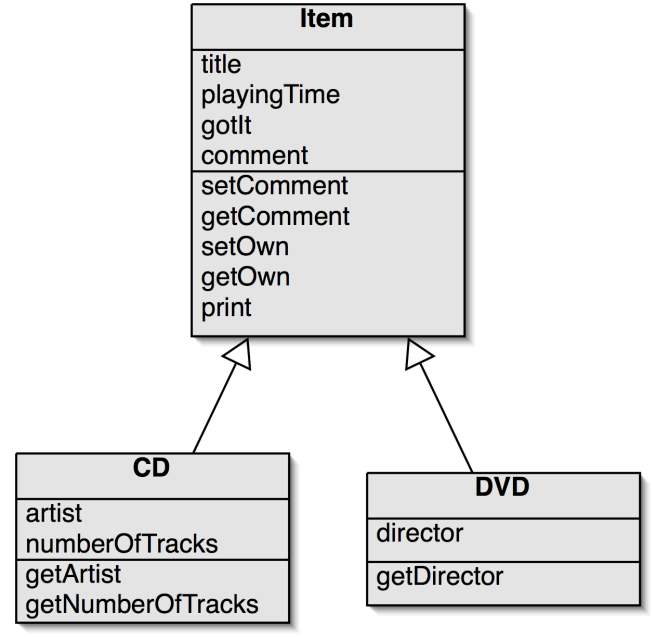
Collections and Inheritance

```
ArrayList<Item> items;
items = new ArrayList<Item>();

items.add( new CD("Nevermind",
                  "Nivana",
                  11, 56) );

items.add( new DVD("Gladiator",
                   "Ridley Scott",
                   155) );


for(Item i : items) {
        i.print();
        int tracks = i.getNumberOfTracks();
        System.out.println(tracks);
}
```



Yes, this is called **substitution**. An Item reference can store any sub-class of Item

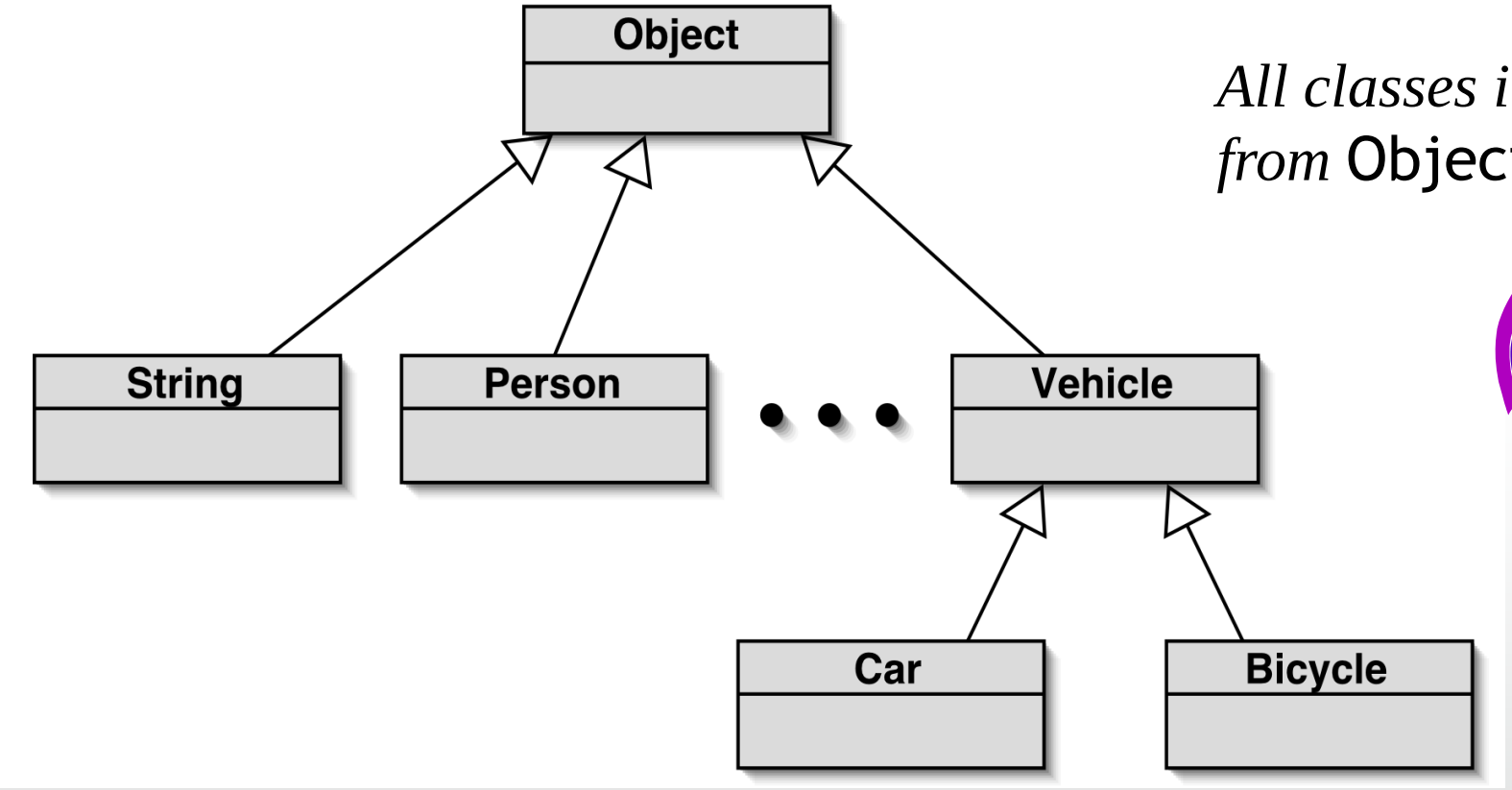
No, because once its stored in an Item reference java will forget if it's a CD or a DVD. This wont compile as Item has a print method but not getNumberOfTracks

The Object Class

Inheritance is a Core Part of Java

- Where is the ‘default constructor’ defined?
- How about the .equals() method,
  - it’s not a part of the String class
- How come we can pass any object to System.out.println() ?
- Where are these magical methods defined?

The Object Class



All classes inherit from Object.

The Object Class

- You've been dabbling with inheritance the whole time
- You don't need to extend the Object class (you can if you really want to) as **every** object extends the Object class (unless it extends something else)
- Look Object up in the API

Summary

- Inheritance and Code Duplication
  - Super and sub-classes
  - Inheritance hierarchies
- Inheritance and Encapsulation
  - Constructors and super()
- Inheritance, References and Collections
  - Substitution
- The Object Class
  - Inheritance at the core of Java